SERVERLESS · COST · ARCHITECTURE

My $8,000 Serverless Mistake

When the cloud bill is the real exception handler

Kevin Griffin · 16× Microsoft MVP · .NET + Azure



I have a severe Azure bias.

ROADMAP

What we'll cover

01 The promise of serverless

…and what it actually is under the hood.

02 Where it shines — and where it bankrupts you

03 The real bills I've paid

04 The fix, and how to avoid the club

Right tool + guardrails.

01

The promise of serverless

“

“It’s just someone else’s servers.”

MENTAL MODEL

Scale-out = more meters

- **More load spins up more hosts**

Each host runs your functions in parallel.

- **Every instance is its own meter**

Hold onto this — it detonates in the $8k story.

- **Trigger ⇒ Action**

Blob / Queue / HTTP triggers invoke your functions.

The Azure Functions plans

Plan	Billing model	RAM / disk	→0?
Consumption (legacy)	GB-seconds + calls	1.5 GB / ~0.5 GB	Yes
Flex Consumption	Exec-time + baseline	up to 4 GB	Yes
Elastic Premium EP1	**Per-sec × instances, ≥1 warm**	**3.5 GB / 250 GB**	**No**
Dedicated	Flat — pay for the plan	up to 256 GB	No
Container Apps	You set min/max replicas	you choose	min=0

“

I didn’t move to EP1 for scale. I moved for RAM and a hard drive.

THE FINE PRINT

What EP1 really bills

- **Per second, at a premium rate**
- **Across every instance it scales out to**
- **Never below one always-warm instance**

“More predictable” ≠ cheaper for continuous work.

02

When serverless shines

GREAT FIT

Serverless at its best

01 Short-lived executions

Seconds to minutes; ~10-min ceiling.

02 No application-host ceremony

Triggers + automatic retries, free.

03 Burstable scale, scale-to-zero

Near-zero at idle, knob to 11 on demand.

“

100,000 items on a queue — drained in minutes.

THE BUILT-IN WARNING

…but SQL became the bottleneck

- **Functions scaled to 11**

SQL connection limits did not.

- **You scale the part you control**

…and stampede the part you don't.

03

When it’s the wrong tool

FAILURE MODE 1

Dependencies can't scale

- **Database connection caps**
- **Third-party API rate limits**
- **Serverless stampedes them**

Remember SQL choking while Functions scaled out.

TAMING IT

Put a governor on it

01 Cap the scale-out / concurrency

02 Batch the work

N items per invocation, not one-per.

03 Queue-based load leveling

The queue is your shock absorber.

“

When all you have is a Function, everything looks like a trigger.

FAILURE MODE 2

High-compute, high-memory, long-running

- Nothing downstream to overwhelm — still a bad fit
- You climb to an expensive always-warm plan for the resources
- Burst pricing works against you

This was my exact profile.

04

The $8,000 mistake

THE WORKLOAD

Continuous, memory-hungry

- Pulled data from a third-party ticketing API, then processed it
- Scheduling forced it to run continuously — a constant grind
- Memory-heavy by nature

THE TRAP

OOM → the easy button

- Symptom: memory limits, OutOfMemory crashes on Consumption
- Fix: a teammate upgraded Consumption → EP1. Crash gone.
- Technically correct. Completely reasonable. The trap.

“

We didn’t fix a problem. We converted a crash into a 24/7 subscription to burning compute.

~$8,000

Compute quietly burned in the first month or two

A continuous workload on a meter that never stops.

“

It wasn’t a bug. The system did exactly what we told it to.

THE TELL

We did it again — for ~~disk~~

- A different app outgrew its disk allocation on deploy
- Cleaning old deploys was fiddly and unreliable
- So we reached for EP1 again. That's the hammer.

Modern fix: run-from-package.

05

It’s not just Azure

The AWS spike — AWS Lambda (US West / Oregon)

Billing period	Bill total	Lambda	GB-seconds
Jan 2025 (normal)	$42.27	$25.13	~1.4M
Feb 2025 (spike)	**$6,654.14**	**$6,613.76**	**~396M**
Mar 2025 (recovered)	$207.71	$181.75	~10.7M

“

Requests barely doubled. Compute went up ~283×.

SAME MISTAKE, DIFFERENT CLOUD

Resource × time, again

- GB-seconds = memory × duration — not request count
- A contractor’s config change left it running away
- Code review catches bugs. It does not catch cost regressions.

“

AWS forgave it — ~99% credited back. That was luck, not a strategy.

06

The fix: containers

WHERE IT LANDED

Azure Container ~~Apps~~ + Container Instances

- Cheaper-per-unit machines for work that runs 100% of the time
- We built our own framework: cron + queue / Service Bus triggers
- Still scales — start at 1 instance, grow to 2, 3, more

A fleet of containers for a fraction of one EP1.

EP1 vs. Container Apps

Elastic Premium (EP1)

Per-second, always-warm

Premium rate, even at idle

No cost ceiling you internalized

Azure Container Apps

Right-sized, cheaper per unit

Bill matches actual use

Min/max replicas you choose

−54%

~$8,000/mo → ~$3,650/mo

~$53K/year back in the budget — same workloads, durable 4+ months.

TAKE-HOME

Should I go serverless?

01 Finishes in seconds-to-minutes?

02 Low / moderate CPU & memory?

03 Dependencies survive max scale-out?

04 Bursty, not steady? And costed at peak?

05 Budgets + alerts wired up before go-live?

07

Watch your bill

“

Azure budgets do NOT stop your spending.

THE GOVERNANCE STACK

Anomaly alerts are the one thing

- **Anomaly alerts — catch the spike that day, not month-end**

  Would have caught BOTH stories.

- **Budgets — they alert; they don't stop**
- **Action-group automation — the actual brake**
- **Cost Analysis as a habit + tagging**

MODERN PRACTICE

AI-backed daily cost review

```
# Claude Code + Azure CLI, ~daily

az costmanagement query \\
  --type ActualCost \\
  --scope /subscriptions/<id>

# → plain-English findings + fixes
# the $53K report came from this
```

CONCLUSIONS

What to take with you

- Serverless is amazing — for the right problems
- Avoid “everything is a nail” (RAM, disk, just bump the plan)
- Watch your bill — watching ≠ stopping; build the brake
- Two clouds, one lesson: it does exactly what you tell it

“

An $8,000 mistake. A $53,000-a-year fix.

Let’s keep talking.

consultwithgriff.com

- @1kevgriff
- linkedin.com/in/1kevgriff
- github.com/1kevgriff
- youtube.com/@consultwithgriff
- @consultwithgriff.com

